fellow mathematician G H Hardy came to visit and mentioned that he'd arrived in a taxi numbered 1729, and he thought the number was "dull". Ramanujan begged to differ!)

In mathematics, the binary, octal, and hex representations of *1729* are, respectively, 11011000001_2, 3301_8, and $6C1_{16}$. Here's the PHP syntax for them:

```
$binRamanujan = 0b11011000001;
$octRamanujan = 03301;
$octPHP8_1Ramanujan = 0o3301; // This notation was added in PHP 8.1
$hexRamanujan = 0x6c1;

var_dump($binRamanujan);
var_dump($octRamanujan);
var_dump($octPHP8_1Ramanujan);
var_dump($hexRamanujan);

Output:
int(1729)
int(1729)
int(1729)
int(1729)
```

Here we can see that binary numbers (Base 2) must be prefixed with `0b`, octal (Base 8) with `0` (or `0o` for PHP 8.1 or later), and hex (Base 16) with `0x`. One interesting thing to note is that in all cases the output is a decimal integer. The same number is represented in different ways syntactically but does not differ internally: a (32-bit) four-byte integer or (64-bit) eight-byte integer (depending on processor architecture). When using non-decimal literals, take care to ensure valid digits are used. Here is what happens if we use digits that are out of bounds:

```
var_dump(0b01012); // Parse error: syntax error, unexpected integer "2" ...
var_dump(0o778); // Parse error: syntax error, unexpected integer "8" ...
var_dump(0xFFG); // Parse error: syntax error, unexpected
identifier "G" ...
```

The specific error emitted by each line is given in the code comments. In all cases it's a fatal syntax error but easily caught by a linter.